

Anti-Fragile Agentic Workflow (AFAW)

A deterministic, multi-agent methodology
for AI-assisted software development

Agustin Diaz-Cano

MS Candidate, Information Systems Engineering,
National Technological University (UTN), Argentina

ORCID: 0009-0001-4336-490X
adiazcano@frba.utn.edu.ar

White paper • Version 1.0 • September 29, 2026

Abstract

Coding agents based on large language models can produce code faster than a person can review it. Running several of them on one repository adds failure modes that a single-agent workflow does not have: agents working on the wrong branch, collisions on shared context files, claims of results that were never measured, and tests that pass whatever the code does. This paper describes the Anti-Fragile Agentic Workflow (AFAW), a repository-level methodology and boilerplate for AI-assisted development with several agents in parallel. AFAW rests on one division of labour: *the AI decides, the engine measures*. Agents generate code; deterministic tools (tests, type checkers, mutation testing, continuous integration) decide whether it is acceptable; a human approves every merge. The method combines per-task branch isolation, task-scoped state files merged by continuous integration, strict test-driven development, validation in parallel cloud jobs, mutation testing restricted to modified files, and enforcement layers that do not depend on the prompt. The reference implementation targets a Python backend; the stack-specific rules are meant to be adapted to other stacks. The rules were derived from failures the author observed in practice. This paper states the method, maps each failure mode to a mechanism and to its residual risk, lists the limitations, and proposes an evaluation protocol. It reports no performance measurements: whether the method improves outcomes remains to be measured.

Keywords: AI-assisted development; coding agents; multi-agent workflows; continuous integration; mutation testing; test-driven development; large language models.

1 Introduction

Coding agents such as Claude Code, Gemini-based tools and Cursor can read a repository, edit files, run commands and open pull requests. A developer can start several of them at once, each in its own terminal. The bottleneck then moves from writing code to keeping the work coherent and verifying it: who is working where, which state is current, and whether the code does what the agent says it does.

Advice for working with coding agents is mostly given as tips for prompting one agent. This paper addresses a different question: what repository-level rules, files and checks keep *several* agents from interfering with each other and make their output verifiable by tools that do not share the model's blind spots?

The paper's contributions are limited and explicit:

1. A catalogue of seven failure modes observed while working with several agents on one repository (Section 2).

2. The method itself: a set of rules, state files and checks (Section 4), whose full rule set is given in Appendix A.
3. A mapping from each failure mode to the mechanism that addresses it and to the risk that remains (Section 5).
4. An evaluation protocol that would let others test the method against a baseline (Section 7).

The reference implementation targets a Python backend, and part of the rule set names Python tools. The method itself does not depend on the language: the stack-specific rules are meant to be re-written for another stack (Section 4.8, and the stack note in Appendix A).

AFAW is a practitioner methodology. It was distilled from the author's experience and has not been validated in a controlled study. No claim about speed, cost or defect rates is made here.

2 Failure modes observed

The failure modes in Table 1 were observed by the author while working with coding agents. They were not logged systematically, so their frequency is unknown, and the list reflects one person's experience.

ID	Failure mode	What it looks like in practice
F1	Wrong-branch work	An agent starts editing files on whatever branch is checked out and collides with work that other agents are doing.
F2	Shared-state conflicts	Two agents update the same shared context file (a last-context or pending-tasks file) and produce merge conflicts.
F3	Unmeasured claims	An agent reports a result (coverage, a passing suite, a speed-up) that it never measured.
F4	Vacuous tests	Tests pass regardless of the behavior of the code they are meant to check.
F5	Heavy local runs	Full test suites launched on the developer's machine block the workflow.
F6	Unreviewed integration	Nothing prevents a direct push to the main branch or a merge without human approval.
F7	Unconstrained side effects	In systems that embed an LLM, a model output triggers a write with side effects without independent validation.

Table 1. Failure modes addressed by AFAW.

3 Design principles

P1. The AI decides, the engine measures. Every number (coverage, metrics, risk) comes from a deterministic function, never from an estimate, a rounding or an extrapolation by the model. The model proposes; tools measure.

P2. Determinism. The same input yields the same numbers. Two identical runs that differ are treated as a bug.

P3. Parsimony. Use the simplest mechanism that solves the observed problem, and add a heavier one only with evidence that the simple one fails. Do not introduce opaque components (for example a trained classifier) that then have to be monitored.

P4. Human authority. Agents propose and structure changes. A human approves every pull request and decides every merge.

P5. State as code. Project state and agent context are versioned files in the repository, so they are diffable, reviewable and reproducible.

P6. Fail loudly, do not fabricate. If a check cannot run, it returns a real error, never an empty result that looks like a pass.

P7. Advice is not a guarantee. Instructions in a prompt can be ignored or misread. Rules that matter are also enforced by mechanisms outside the model.

4 The method

Figure 1 shows the whole flow. The human opens one isolated terminal per agent and gives each a role. Each agent works on its own branch, follows test-driven development, and pushes. Continuous integration (CI) validates in the cloud. The human approves the pull request, and CI regenerates the shared state through a second pull request that the human also approves.

4.1 Roles and session start

In its first message, the agent asks the human for its role (ROL :). It then reads the roadmap file (PENDING.md) and the last-context file of that role, or the global last-context file if the role has none. Before touching any file it runs `git status` and `git branch`, because other agents are working in the same repository. It never works on the branch that happens to be checked out: it creates a new, isolated branch first.

4.2 Task isolation and the life of a task

Each task gets its own branch, named after the task with a prefix (feat/, fix/ or chore/) and a clear description. Work is committed in atomic units: one feature, one commit, in the Conventional Commits format [4]. The agent never commits or pushes to main. Figure 2 gives the twelve steps from the first question to the deletion of the branch.

At the end of a task the agent updates the affected documentation and its own task files, and records them in a separate documentation commit that contains documents only. The agent then asks whether to create the pull request, unless the human has already asked for it. The pull request carries a title, a description and the link to the CI run. The human approves it manually, and merging requires an explicit human directive.

4.3 Isolated state and context

Agents never edit shared state files. Each agent writes only two files: its delta (context/tasks/task_NNN_context.json) and its task file (state/tasks/task_NNN.json). Three kinds of shared file are generated and never edited by hand:

- state/pending.json
- context/global/lastcontext.json
- context/roles/*/lastcontext.json

After a merge, a CI script (scripts/merge_state.py) regenerates them and opens a pull request that the human approves.

A second CI script (scripts/check_task_delta.py) rejects a pull request that changes code without a valid delta listing those files in a `files_touched` field. Which folders count

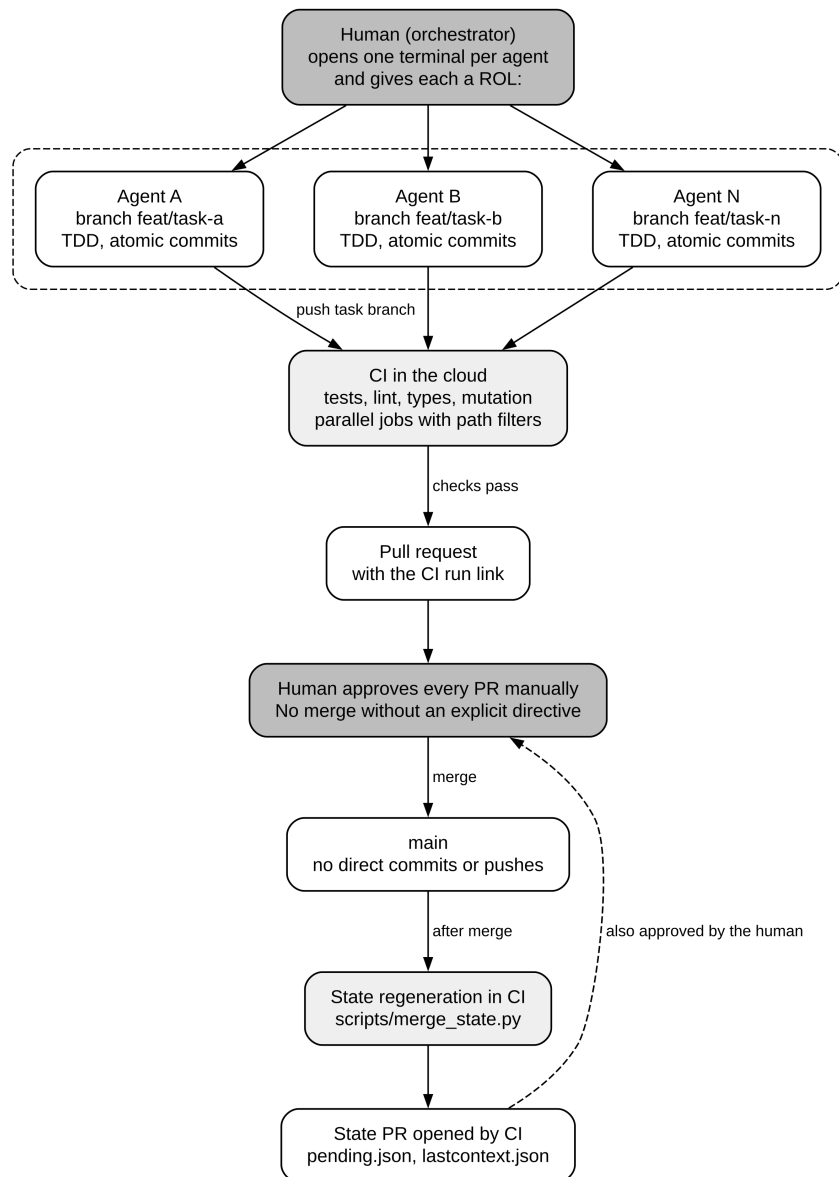


Figure 1. Overview of AFAW. The three agents are isolated, one terminal each. White: agent action; grey: human; lightest grey: CI or deterministic check; medium grey: state files; darkest grey: blocked.

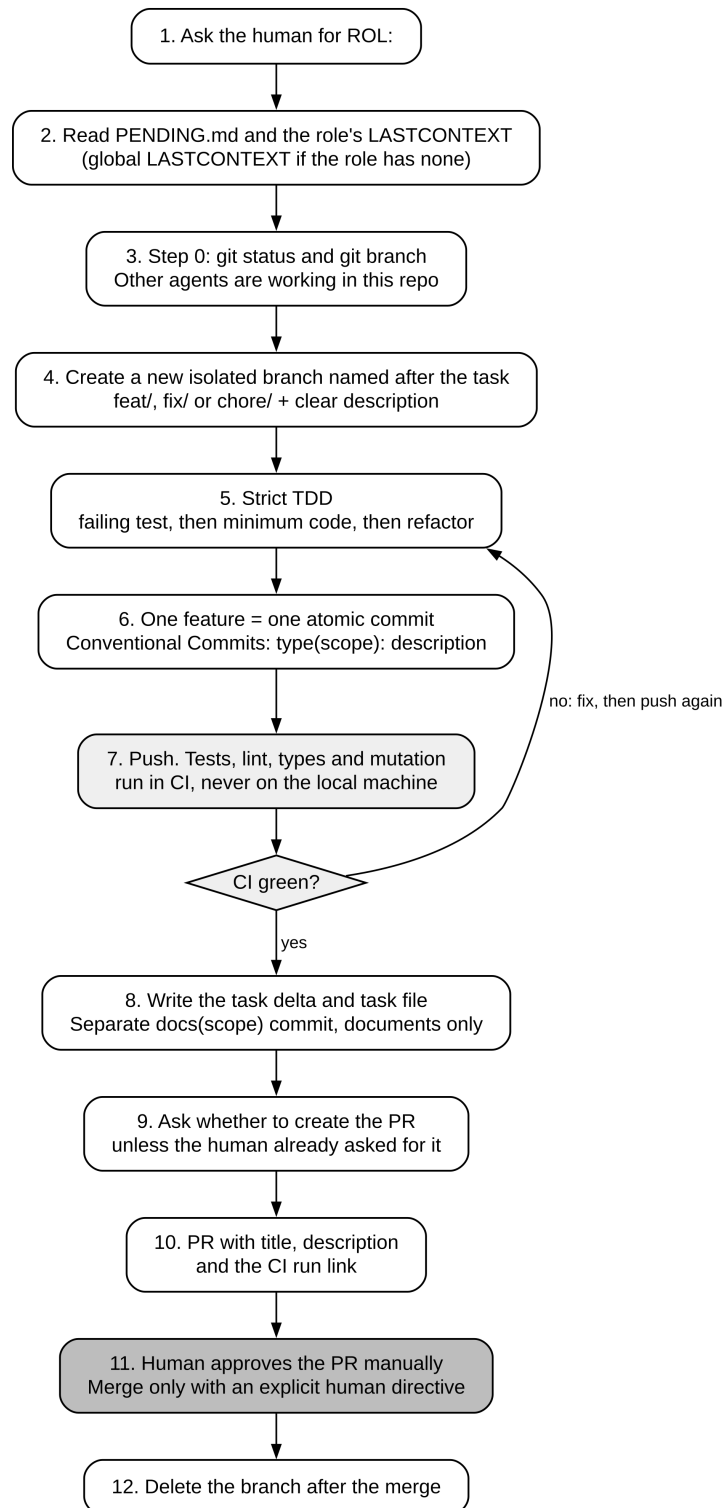


Figure 2. The life of a task, from asking for a role to deleting the branch after the merge.

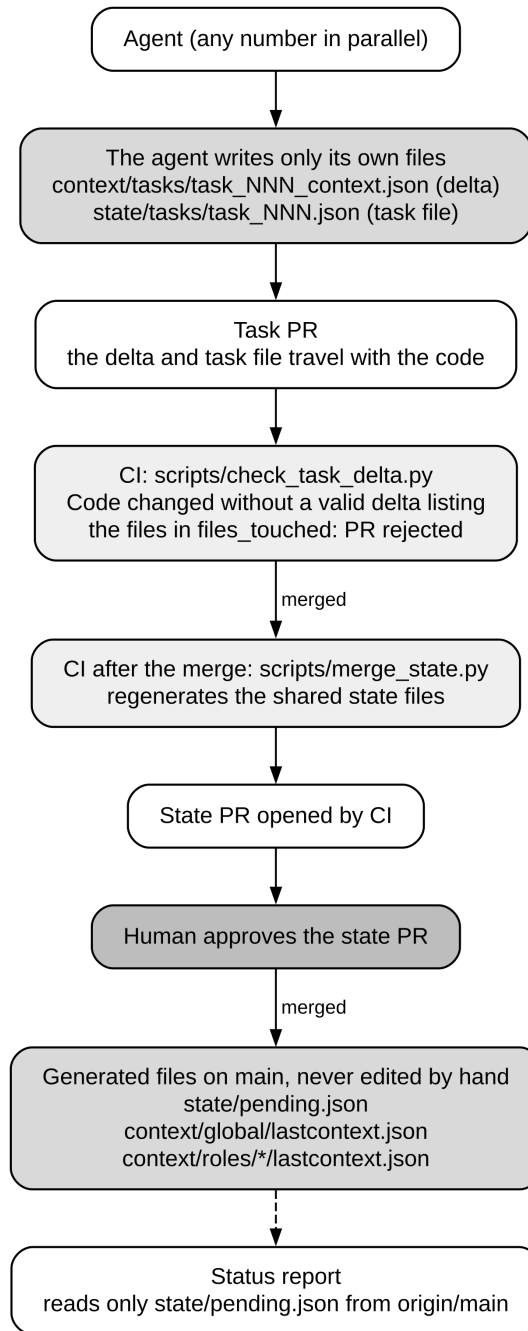


Figure 3. State and context. The agent writes only its own delta and task file; CI checks the delta, regenerates the shared files and opens a state pull request.

as code is set per project in `state/config.json`. Figure 3 shows the flow. Serializing the regeneration runs (for example with a CI concurrency group) prevents the bot pull requests from racing each other; how well this scales with many agents has not been measured.

A status report reads only `state/pending.json` from `origin/main`, because the copy on a working branch may be stale. It is returned as a table with one row per task, a traffic-light status, a difficulty from 1 to 5 and a priority from 1 to 5.

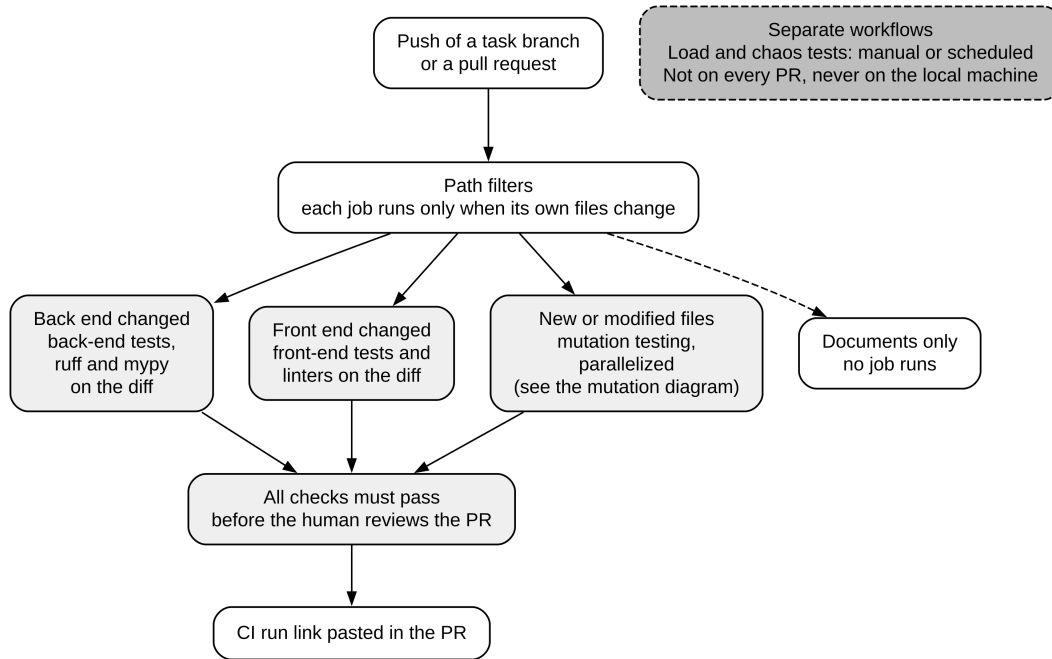


Figure 4. The CI pipeline. Path filters route a change to parallel jobs; load and chaos tests are separate workflows.

4.4 Verification in CI

Development is test-driven in the strict sense: the agent writes a test, confirms that it fails, writes the minimum code that makes it pass, and refactors with the test green. For a bug, the first step is a test that reproduces it and fails. When the task is to add tests to existing code, only the test folder is edited. If a new test fails against the original code, the test is presumed wrong unless there is evidence of a real defect, in which case it is marked as an expected failure with the reason recorded, and the source is not changed to make it pass.

Tests, linters, type checks, mutation testing and builds are launched in CI, not on the developer's machine. Independent jobs run in parallel, and path filters make each job run only when its own files change: back-end changes launch back-end jobs, front-end changes launch front-end jobs, and document-only changes launch nothing. Linters run only on the modified files. Load and chaos tests run as manual or scheduled workflows, not on every pull request. Figure 4 summarizes the pipeline.

Verification rules apply to the checks themselves. A script that reports PASS does not prove that it measured the right thing, so internal consistency (two runs agree) is combined with a known external value. Credentials and configuration are validated before minutes of compute are spent.

4.5 Mutation testing on the diff

Mutation testing [5, 6] alters the source in small ways (mutants) and re-runs the tests. If the tests still pass, no test detected that change. The line may well be executed; no assertion depends on it. With N generated mutants, K killed and E equivalent to the original, the mutation score is $K/(N - E)$. Coverage and mutation score measure different things and should be reported separately.

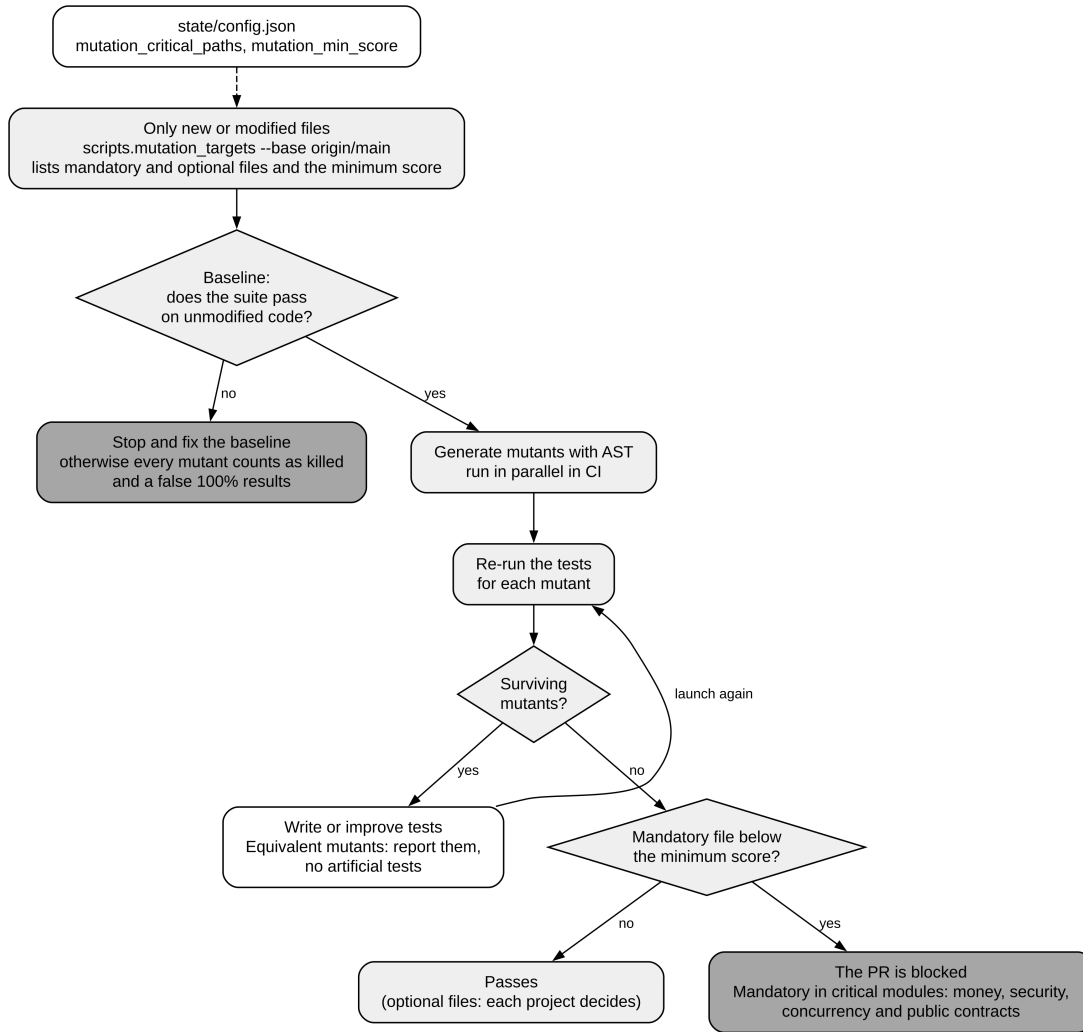


Figure 5. Mutation testing on the diff, with the baseline check and the blocking rule for mandatory files.

AFAW applies mutation testing only to new or modified files, in parallel in CI (Figure 5). Before mutating, the baseline is validated: the suite must pass on unmodified code, otherwise every mutant would count as killed and a false 100% would result. Surviving mutants are fixed by writing or improving tests, and the run is repeated. Equivalent mutants are reported rather than covered with artificial tests. The step is mandatory in critical modules (money, security, concurrency, public contracts), where a file below the minimum score blocks the pull request. The critical paths and the minimum score are defined per project in `state/config.json` (`mutation_critical_paths` and `mutation_min_score`).

4.6 Human authority and enforcement layers

The rules “never commit or push to main”, “never merge without an explicit human directive” and “the human approves every pull request” are written in the agent instruction file. Because that file is advice (P7), they are also placed under mechanisms that do not depend on the model (Figure 6). Hooks intercept commands before they run; one example is a hard-deny rule that lets `PENDING.md` be edited but never deleted or emptied. CI checks block a pull request that fails tests, lint, types, mutation or the task-delta check. Branch protection on `main` makes the server reject a direct push. The human approval is the last step.

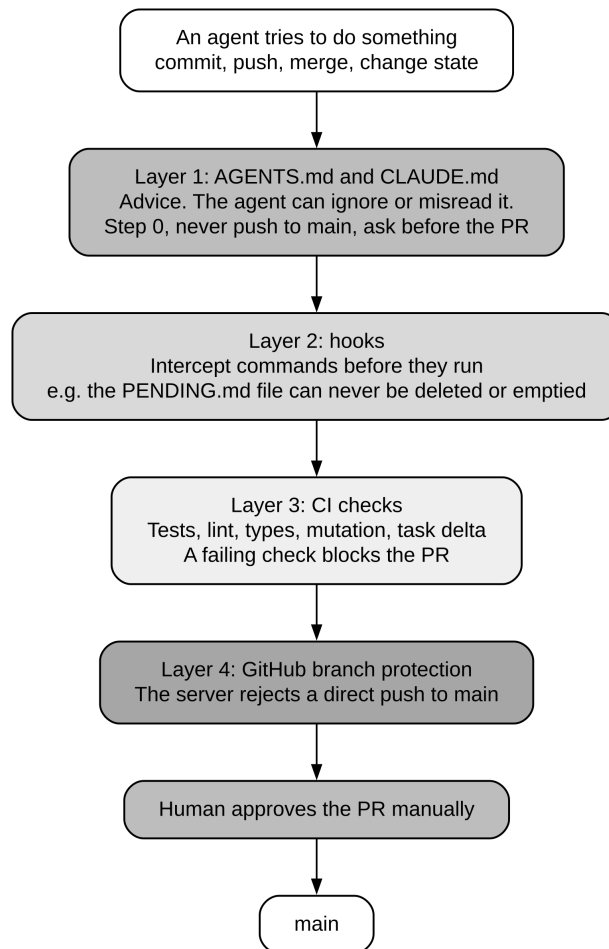


Figure 6. Control layers. The layers are shown in a rough order; in practice each acts at a different moment.

Hooks are a best-effort filter, not a security boundary: a command can be phrased in a way a pattern misses. Branch protection is the barrier that does not depend on the agent.

4.7 A module for systems that embed an LLM

When the product itself contains LLM agents (F7), AFAW adds rules that follow the same principles. The LLM never executes tools with side effects: only deterministic code does, and every write tool is idempotent by a request identifier. LLM output is never trusted: it is revalidated on the server with strict schemas and business limits. The user-facing agent has minimum privilege, and a prompt-injection filter runs before a message reaches it. Every component is declared fail-open or fail-closed; guards and judges fail closed, while context retrieval and tracing fail open. Each tool call is authenticated and authorized on the server, and every invocation, including denied ones, is written to an audit log with personal data masked. Model judges are evaluated on the production code, and false approvals are reported first because they are the dangerous error.

4.8 Adapting to other stacks

Most rules are stack-independent. The rules on code standards, architecture, concurrency and messaging, debugging, and infrastructure (Appendix A, sections 4–10 and 14–16) assume a Python backend with asynchronous I/O, a relational database, a message broker, Terraform and the Model Context Protocol. For other stacks they should be adapted or removed.

5 From failure mode to mechanism

Table 2 links each failure mode to the mechanism that addresses it and states the risk that remains. The last column matters as much as the second: none of the mechanisms removes its failure mode.

ID	Failure	Mechanism	Residual risk
F1	Wrong-branch work	Step 0 (<code>git status</code> , <code>git branch</code>); one branch per task; one isolated terminal per agent.	Step 0 detects a shared working directory but does not prevent it. Agents sharing one working tree can still overwrite each other's uncommitted files. The rule itself is only advice.
F2	Shared-state conflicts	Task-scoped delta files; shared state regenerated by CI; delta check on every pull request.	Bot pull requests may pile up or conflict with many agents. Not measured.
F3	Unmeasured claims	Rule “never claim a result that was not measured”; CI as the gate; run link in the pull request; fail loudly.	A person must still follow the link; a compliance failure by the agent is caught only if someone checks.
F4	Vacuous tests	Test first and confirm failure; mutation testing with baseline validation; equivalent mutants reported.	Mutation score is not correctness. Tests and code written by the same model can share blind spots.
F5	Heavy local runs	Everything heavy in CI, in parallel, filtered by path.	Queue and start-up latency; quota of CI minutes on private repositories; slower feedback for very small checks.
F6	Unreviewed integration	Rules, hooks, CI checks, branch protection, human approval.	Hook patterns are best-effort. A solo developer cannot require an approving review from a second person.
F7	Unconstrained side effects	Deterministic execution layer, strict revalidation, least privilege, audit log.	Schema and validation bugs; attack surfaces beyond the ones anticipated.

Table 2. Failure modes, mechanisms and residual risks.

6 Limitations and threats to validity

- **No measurement.** This paper reports no data on speed, cost, tokens or defects. The benefits of the method are hypotheses.
- **Single origin.** The rules come from one practitioner's experience with a small set of projects. The failure modes may not represent other teams or codebases (selection bias).
- **Prompt compliance.** An instruction file does not guarantee that a model follows it. How often the rules are violated, and how often the mechanisms in Section 4.6 catch a violation, has not been recorded.
- **Mutation testing.** Mutation score measures how sensitive the tests are to changes in the code. It does not show that the code meets its requirements, and equivalent mutants

cannot be killed by any test. The AST-based operators are limited to what the engine implements.

- **Shared blind spots.** If the same model writes the code and the tests, deterministic checks can pass on a wrong interpretation of the requirement. Independent tests or a second source of ground truth are needed to detect this.
- **CI-only design.** Sending all checks to CI trades feedback latency and CI minutes for a clean local machine. A hybrid (fast checks locally, heavy checks in CI) may be better and has not been compared.
- **State regeneration under load.** The regeneration pull requests have not been tested with many agents in parallel.
- **Human review remains necessary.** The method reduces how much code a person must read, but intent, design, security and the tests themselves still need human judgement.
- **Stack specificity.** Part of the rule set assumes a Python backend (Section 4).
- **Model drift.** Agent behavior depends on the model and tool version. Any measurement should record both and the date.
- **On the name.** “Anti-fragile” follows Taleb’s term for systems that gain from disorder [8]. Here it is used informally: failures caught by deterministic checks are turned into stronger tests and rules. This paper does not show that the method gains from stress in Taleb’s formal sense.

7 Proposed evaluation protocol

The claim that AFAW helps should be tested, not assumed. This section proposes a design that others (or the author) can run.

Research questions.

RQ1 Does state isolation with CI-regenerated context reduce merge conflicts and wrong-branch commits compared with agents sharing context files?

RQ2 Does the mutation gate reduce escaped defects and vacuous tests?

RQ3 What does the method cost in elapsed time, tokens and CI minutes?

RQ4 How much human review effort does each pull request need, and what does the reviewer find?

Design. Run the same task set under matched conditions: (A) a baseline with minimal agent instructions; (B) full AFAW; and ablations of B that remove one component at a time (state isolation, the mutation gate, CI-only execution). Fix and record the model, the tool version and the date. Randomize the order of tasks and repeat runs, because agent output is not deterministic. Tasks can come from the project’s own backlog. Public issue-resolution benchmarks with hidden tests, such as SWE-bench [7], can supply an independent ground truth that the agents’ own tests cannot.

Metrics. Table 3 lists the measures and where each can be read from.

Reporting. Publish the raw data and the exact instruction files. Report intervals and not only means. Report false approvals first and negative results as well. State the model, tool version and dates so that the result can be read against later models.

Metric	Definition	Source
Merge conflicts per PR	Conflicts that need manual resolution	Git history
Wrong-branch commits	Commits made outside the task branch, or blocked attempts	Hook and server logs
Time to green CI	Task assignment to first fully green run	CI API
Tokens per merged task	Model tokens spent until merge	Agent usage logs
CI minutes per merged task	Compute minutes billed	CI billing data
Escaped defects	Failures found by hidden tests or after merge	Hidden tests, issue tracker
Mutation score and coverage	Reported separately for merged files	Mutation engine, coverage tool
Human review minutes per PR	Time a reviewer spends before approving	Reviewer log
Rule violations	Times an agent broke a rule, and by which mechanism it was caught	Hook, CI and protection logs

Table 3. Proposed metrics.

8 Related work

AFAW builds on established practices rather than replacing them. Test-driven development [2] provides the red–green–refactor loop. Mutation testing goes back to DeMillo, Lipton and Sayward [5], and Jia and Harman [6] survey its development. Short-lived branches from an always-deployable main branch follow GitHub Flow [3], and commit messages follow the Conventional Commits specification [4]. Instruction files for coding agents are now a common convention; the AGENTS.md format [1] is one example, and CLAUDE.md is another. The term “anti-fragile” is Taleb’s [8]. SWE-bench [7] evaluates language models on real repository issues with hidden tests and is relevant to the evaluation protocol.

This paper does not survey the fast-moving literature on coding agents. Related work is limited to the sources on which the method directly depends.

9 Conclusion

AFAW is a set of rules, state files and checks for working with several coding agents on one repository. Its central choice is to let models generate and let deterministic tools verify, with a human approving every merge. It targets seven observed failure modes and states the risk that remains for each. It is a hypothesis about how to work, not a demonstrated result. The protocol in Section 7 is offered so that the hypothesis can be tested, and refuted if the data say so.

Declarations

Availability. The reference boilerplate (agent instruction file, state layout, scripts and diagrams) is distributed as a repository template; its licence is stated in the repository.

Use of AI assistance. An AI assistant (Claude, Anthropic) was used to help draft and edit this paper and to produce the diagrams. The author defined the method, reviewed the text and is responsible for its content.

References

- [1] AGENTS.md. *A simple, open format for guiding coding agents*. <https://agents.md/> (accessed 29 September 2026).
- [2] K. Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.
- [3] S. Chacon. GitHub Flow. Blog post, 31 August 2011. <https://scottchacon.com/2011/08/31/github-flow/>.
- [4] Conventional Commits. *Conventional Commits Specification, version 1.0.0*. <https://www.conventionalcommits.org/en/v1.0.0/>.
- [5] R. A. DeMillo, R. J. Lipton and F. G. Sayward. Hints on test data selection: help for the practicing programmer. *Computer*, 11(4):34–41, 1978. doi:10.1109/C-M.1978.218136.
- [6] Y. Jia and M. Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi:10.1109/TSE.2010.62.
- [7] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- [8] N. N. Taleb. *Antifragile: Things That Gain from Disorder*. Random House, 2012.

A Complete rule set

This appendix reproduces the complete instruction file (AGENTS.md, identical to CLAUDE.md) that the agents receive, section by section, as used by the author. It is the source of truth for the method; the body of the paper summarizes it. Two edits were made for typesetting: the traffic-light emoji of section 3 are written as the words green, yellow and red, and the file’s opening heading is omitted.

Stack note. The reference implementation of these rules targets a Python backend (asynchronous I/O, pytest, mypy, a relational database, a message broker, Terraform and the Model Context Protocol). The stack-independent core is sections 0–3, 11–13 and 17–19. Sections 4–10, 14–16 name concrete Python tools and infrastructure, and should be re-written for another stack while keeping the intent of each rule.

Rules reusable in any project, unified from two real AGENTS.md/CLAUDE.md files plus our own rules. Copy to the repo root as CLAUDE.md and AGENTS.md (keep both identical).

0. LANGUAGE — ALWAYS SPEAK ENGLISH

ALWAYS RESPOND IN ENGLISH. Every reply, explanation, commit message, PR title/description and code comment is written in English, regardless of the language the user writes in.

1. Session start and context

1. **In the first message, always ask the human for ROL:** (which role this agent has in the project).
2. Besides reading the context files (PENDING.md), look for that role’s LASTCONTEXT: if it exists, read it; if it does not, read the global LASTCONTEXT.
3. Update the context at the end of every relevant session.

4. **When finishing a task:** update the affected documentation and the status documents (the task's own context file and its state in `state/tasks/`, never the global `LASTCONTEXT`), and make a documentation commit for the task (`docs(scope): ...`), separate from the code commit, with a message saying what was done. That commit contains documents only, so it does not launch tests (section 11).
5. Keep the duplicated instruction files in sync (`AGENTS.md` and `CLAUDE.md` identical): edit both in the same change.
6. Document the status of each phase (done, in progress, roadmap) together with the decisions taken and their date.
7. **Generated files are not edited by hand:** `state/pending.json`, `context/global/lastcontext.json` and `context/roles/*/lastcontext.json` are regenerated by CI (`scripts/merge_state.py`), which opens a PR that the human approves. The agent only writes its delta (`context/tasks/task_NNN_context.json`) and its task file (`state/tasks/task_NNN.json`). CI rejects a PR that changes code without a valid delta that lists those files in `files_touched` (`scripts/check_task_delta.py`); which folders count as code is defined per project in `state/config.json`.

2. Role and way of working

1. Act as a senior engineer and QA specialist: SOLID, Clean Architecture, small, modular, typed, testable, production-ready code.
2. **Never claim a result that was not measured.**
3. **Strict TDD (Red → Green → Refactor):** write the test first and confirm it fails; then the minimum code to make it pass; refactor with the test green.
4. Never present a function or endpoint as finished without its test.
5. For a bug, first write a test that reproduces it (it must fail) and only then fix it.
6. Tests for detectors or estimators use synthetic sequences with known noise and a known change point.
7. **Parsimony:** implement the simplest model or detector that solves the problem; use a heavier one only with evidence that the simple one fails.
8. Do not add opaque components (e.g. a trained classifier) that then have to be monitored.
9. Consult the current documentation of a library before implementing with it; do not do it from memory.

3. Project status report

When the project status is requested, **always** answer with a table of pending tasks with traffic light, difficulty and priority:

Task	Status	Difficulty (1-5)	Priority (1-5)
Example: migrate the worker to async	yellow	3	1

1. **Traffic light:** green done or on track without risk · yellow in progress or at risk · red blocked, failing or overdue.
2. **Difficulty:** 1 = trivial, 5 = very hard or uncertain.
3. **Priority:** 1 = most urgent, 5 = least urgent. Sort the table by ascending priority.
4. One row per task; no paragraphs inside the table. If a task is red, give on a separate line the blocker and what is needed to unblock it.
5. Only real project tasks (`PENDING.md`); do not invent tasks to fill the table.
6. **To read the status, read only `state/pending.json` from `origin/main`** (`git fetch` and `git show origin/main:state/pending.json`), because the branch's own copy may be out of date. Do not read the files in `state/tasks/` one by one.

4. Code standards

1. Type hints on every public function, with a short docstring (what it measures/does and which file it writes).
2. `mypy --strict`; no Any unless justified in an inline comment.
3. Every function that performs I/O is `async`, and there is never blocking I/O inside `async` (use `asyncio.sleep`, `httpx.AsyncClient`, `async drivers`).
4. Every blocking subprocess call has a timeout; long-running processes are terminated or awaited; temporary working directories are deleted.
5. Configuration from environment variables with `pydantic-settings`; never hardcode secrets or connection strings.
6. Structured logging (`structlog`); never `print()`.
7. PEP 8 with a maximum line length of 100 characters.
8. No `from module import *`.
9. Do not use a generic `except Exception` without logging or re-raising the specific error.
10. No TODO comments: implement it or ask.
11. Do not add new dependencies without justifying them; prefer the standard library.
12. Single-purpose functions; one file, one responsibility; split a file before it exceeds ~600 lines.
13. Every module has its own test file (`pytest + pytest-asyncio`).
14. Tests run against their own database whose name ends in `_test`; the suite refuses to start otherwise.
15. All code lives in `src/` with absolute imports, so it works the same locally, in tests and in Docker.
16. Sample or demo data is loaded with separate scripts or jobs, never inside a migration.

5. Architecture

1. **The AI decides, the engine measures:** every number (coverage, metrics, risk) comes from a deterministic function, never from an estimate, rounding or extrapolation.
2. **Determinism:** same input, same numbers. Two identical runs that differ are a bug.
3. **Layer separation:** never mix database, routing and AI logic in the same file. The core has no web/CLI/protocol code; the adapters (CLI, server, MCP) are thin and delegate.
4. Repositories only access the database; external HTTP calls go in services or agents.
5. **The AI layer never imports models, repositories or database connections:** all data access goes through tools (MCP).
6. **Mandatory idempotency:** every request carries a unique `request_id`; it is checked before invoking the LLM and, if it exists, the cached result is returned.
7. **Retries with exponential backoff** and a configurable limit on every external call; when exhausted, NACK to requeue or send to dead-letter.
8. **LLM provider factory:** switch provider with an environment variable, without touching business logic. A mock provider for all roles avoids spending in tests and load runs.
9. Scoring/rule modules are pure functions of their inputs, with no access to the database or tools, so that every firing is reproducible and auditable.
10. Log every rule firing (id, inputs, belief degree, final verdict): it is what makes the layer auditable, and it is not omitted to save writes.
11. Observability and drift detection never block or take part in the critical path: they consume already persisted logs and their failure does not affect transactions.
12. Explicitly define each component as fail-open or fail-closed: guards and judges fail closed; context retrieval and tracing fail open. A missing configuration value in a security check fails closed.
13. **Outputs as a contract:** functions return a dict and write JSON to an output folder; subagents communicate through files, not shared memory.
14. **Compact responses:** summaries by default, full data only on request (everything stays in the agent's context and is resent every turn).
15. Do not print to stdout in stdio servers: it is the protocol channel; use stderr or return values.
16. A read-only internal frontend consumes only GET endpoints and never connects directly to the

- database or to internal services.
- 17. Choose the transport deliberately and document it; a global change of transport is discussed first, and an alternative is added as an additional option.
- 18. **The source code is the reference:** when writing tests only `tests/` is edited. If a new test fails against the original code, the test is wrong unless there is evidence of a real bug; in that case `xfail(reason="possible bug: ...")`, never fix the source code to make it pass.
- 19. Work on temporary copies, never in place, when the target repo is the basis of the documented metrics.

6. Agents and LLMs

- 1. **The LLM never executes tools with side effects:** only deterministic code does, and every write tool is idempotent by `request_id`.
- 2. **Never trust the LLM's output:** revalidate it on the server with strict Pydantic schemas (`extra="forbid"`) and business limits.
- 3. The user-facing agent has minimum privilege: no access to the database, the knowledge table or the tool server. It only proposes a structured intent and, if it cannot map the message, asks instead of guessing a field.
- 4. A prompt injection/jailbreak filter before the message reaches the agent, which fails fast.
- 5. Feedback to the user exposes an objective, auditable verdict, never a judge's raw justification, which could leak internal reasoning or business rules. The agent words it but never reinterprets or overrides it.
- 6. Catch any exception in an agent's tool dispatch and return it to the model as an error result: one bad call must not bring down the whole process.

7. Security

- 1. Authenticate every tool call with a bearer token: the server stores only its SHA-256 hash, compares it in constant time and derives `client_id` from the token, never from a client header.
- 2. Per-tool authorization with a server-side allowlist; nothing in the prompt or in retrieved documents can widen access.
- 3. Rate limiting per client and tool, computed from the audit table so it holds across replicas without Redis.
- 4. Audit log of every invocation, including denied ones, with PII masked; if the audit write fails, the tool is not executed.
- 5. Mask PII in trace exports (pseudonymize user ids, redact emails and long numbers); every new PII field is added with its test before being traced.
- 6. One service account per service, with access per secret and never at project level.
- 7. Values that Terraform does not generate are loaded by hand, not through Terraform variables, so they do not end up in the state.
- 8. Normalize secrets (`.strip()`) on both sides of a comparison; beware of newlines in secrets.
- 9. Document exposure analyses (e.g. abuse of a public API) and do not implement a defense without the user's approval.
- 10. Persistence is opt-in and never public on routes that accept arbitrary input; do not store rounded numbers.

8. Concurrency and messaging

- 1. Pessimistic locking (`SELECT ... FOR UPDATE`) plus a `UniqueConstraint` on the id to prevent double processing.
- 2. A recovery sweeper (`FOR UPDATE SKIP LOCKED`) requeues what a crashed worker left half done.
- 3. Publish messages as persistent on durable queues: a transient message is lost when the broker restarts.

4. Safe ack/nack (log and continue if the channel closed) and reconnection with retry, instead of dying.
5. Long-lived consumers use `restart: unless-stopped` and wait for the broker themselves; normal services use a bounded restart as a defense.
6. Close the database transaction before long external calls (MCP, LLM).
7. Use `clock_timestamp()` and not `now()` for audit timestamps: `now()` reflects the start of a long transaction and distorts latencies.
8. In clients with a timeout, open a new session per attempt and do not reuse it across retries.
9. **In-memory state + multiple instances = bug:** locks and rate limiters must be durable (database) if the service scales horizontally.

9. Observability and traces

1. One trace per transaction, with an id derived from the `request_id`, and typed, nested observations.
2. Observation names are an API: stable, verb first, low cardinality; ids and models go in metadata. Renaming is a breaking change.
3. Tracing is fail-safe: it turns off without keys and its failures are logged without changing the result or the ACK/NACK.
4. Tests never send traces: tracing is disabled in the suite and an in-memory exporter is used when asserting on spans. If the test depends on callbacks, use real fake models and not `AsyncMock`.
5. Verify against a real trace after changing the instrumentation.

10. Evaluating models and judges

1. Evaluate the production code, never a pasted copy of the prompt.
2. The labels of the cases are decided by the user: new cases stay pending until he reviews them.
3. Report false approvals first: they are the dangerous error and accuracy alone hides them.
4. Keep context retrieval fixed in evaluations.
5. Real runs cost money and time: use the mock provider to test the flow and pass up-to-date prices with a date.
6. Asymmetric judges from different model families; the tie-break uses a different model from the first judge's to give an independent opinion.
7. LLM verdicts vary between runs: do not repeat the vote to "correct", and cover that gap with deterministic checks.

11. Code tests

1. **Tests are always launched in the cloud** (GitHub Actions or whatever CI the project uses), never on the local machine: running them locally is a waste of time. The same goes for linters, type-check, mutation testing and builds: **everything is launched in CI.**
2. **They are parallelized whenever possible;** they are not queued.
3. Changes in the back end launch only the back-end tests; changes in the front end, only the front-end ones.
4. **Document changes do not launch tests:** only what contains code.
5. Use path filters (`paths/paths-ignore`) in the workflows so that each job runs only when its files change.
6. Load and chaos tests (Locust, killing workers, restarting the broker) are also launched in the cloud (manual or scheduled workflow), never locally, and do not run on every PR because of their cost and duration.

12. Code fragility and coverage (mutation testing with AST)

1. To measure fragility, **AST** is used: the code is intentionally altered (mutants) and the tests are launched again. If the tests still pass, that line was not really covered.
2. Surviving mutants are fixed by writing or improving tests, and AST is launched again, in a loop, until the real coverage needed is met.
3. **AST is launched only on new or modified files**, never on files that did not change.
4. This set of tests is launched **parallelized in the cloud, on GitHub**.
5. Validate the baseline before mutating: the suite must pass without modifying the code, or every mutant would count as “killed” and a false 100% would result.
6. Equivalent mutants (e.g. `round(x, 2) → round(x, 3)`): report them, do not write artificial tests.
7. **Mandatory in critical modules** (money, security, concurrency, public contracts). In the rest it is optional and each project decides. A mandatory file below the minimum score blocks the PR.
8. The critical folders and the minimum score do not live in this file, because they change from project to project: they are defined in `state/config.json` (`mutation_critical_paths` and `mutation_min_score`). `python -m scripts.mutation_targets --base origin/main` lists, among the modified files, which are mandatory and which are optional, and the minimum score. The mutation engine and its CI job are specific to each project and call that script.

13. Verification

1. Verify after any engine change against documented baseline values, in CI.
2. **A script that says PASS does not prove it measured the right thing:** combine internal consistency (two runs agree) with a known external value.
3. **Fail loudly, do not fabricate:** if a check could not run (missing dependency, credentials), return a real error and never an empty result that looks like a pass.
4. **Fail fast:** validate credentials and configuration before spending minutes of compute.
5. Never leave fixtures or verification reference files inside the project after using them.
6. Validate with real end-to-end claims in the deployed environment, not only with tests.

14. Debugging, load and chaos

1. On a failure between containers, isolate the transport from the application: write the minimal script that exercises only that connection and run it with `docker exec` inside the failing container.
2. If the script passes, the bug is in the app’s control flow or exception handling, not in Docker, DNS or the network.
3. **A missing dependency $\neq$ a missing config:** both look like an opaque 500; check the server traceback before assuming.
4. Document postmortems.
5. Validate with real load (Locust) and with chaos: kill workers, restart the broker and verify invariants in SQL (zero duplicates, zero lost, nothing stuck).
6. Measure scaling by pausing the workers (`docker pause`), not with `stop/start`.
7. Watch out for heavy healthchecks: they can keep an idle broker at high CPU.

15. Environment and dependencies

1. **Never call `subprocess.run(["pytest", ...])` with a “bare” command name:** use `[sys.executable, "-m", ...]` to pin the interpreter and not depend on the ambient PATH.
2. Parallel checkouts/worktrees: a script installed in editable mode may point to another checkout; verify with `pip show` before trusting a run.
3. Pin exact versions of coupled packages (e.g. `fastapi/starlette`) and reinstall clean if they get out of sync.
4. Document environment variables in a table with description and default value.

16. Infrastructure and deployment

1. One Docker image per service, with production dependencies only and a non-root user.
2. Tag images with the commit, never with latest.
3. All infrastructure as code (Terraform) with versioned state.
4. Scripts to turn the demo environment off and on, to control costs.
5. Update the infrastructure documentation in the same PR as any infrastructure change.
6. Do not change broker or provider without confirming, and document each decision with its date.

17. When to pause and confirm

1. Confirm with the user before touching:
 - DELETE or UPDATE without WHERE.
 - Migrations that drop columns or tables.
 - The ACK/NACK logic.
 - Public tools (renaming or removing) or write guards.
 - Thresholds of a security guard (lowering them) or measurement operators/thresholds without re-measuring and updating the docs in the same change.
 - Fixtures that affect all documented numbers.
 - .env and secrets.
 - Model, router or tool-registry files (list the impact before deleting or moving).
 - Published numbers without a real measurement behind them.
2. Before a migration, verify the current revision (a Lembic `current`) and confirm the target with the user; this applies equally to the cloud database.
3. Protect the personal roadmap file (PENDING.md) with a hard-deny hook: its content can be edited, but the file can never be deleted or emptied.
4. When refusing an action, always state the correct alternative in the same reply.

18. Linters, branches, commits and PRs

Linters

1. Linters and type-check (ruff, mypy, and the front-end ones) run in CI, not locally.
2. They are launched only on what was modified (the diff), and only the linter for the side that changed (back end or front end).
3. They do not run on document-only changes.

Branches and commits

4. GitHub Flow: an always-deployable main and short branches. **Step 0, always:** before modifying any file, check the branch and the working tree (`git status` and `git branch`), because multiple agents are working on the project. Never start working on the current branch by default: first create a new, isolated branch for the task.
5. **The branch name represents the task** (feat/, fix/, chore/ + a clear description, e.g. feat/fuzzy-scoring-layer).
6. **Atomic task system:** one feature = one commit with a representative name.
7. Conventional Commits: `type(scope): description`.

Pull requests

8. **Never commit or push to main.**
9. **Never merge without an explicit human directive.**
10. **The human always approves the PR manually.**
11. **The agent asks whether to create the PR, unless the human already asked for it in the instruction.**

12. When creating a PR, deliver the title and a comment with the description in the reply, and also write them in the PR if the environment allows it.
13. The checks (linters, tests, AST) run in CI; paste the link to the run in the PR, not a local output.
14. If a PR changes a measured number, update README/docs in the same PR.
15. After the merge that the human decides, delete the branch.

19. Token budget

1. Short replies: tables and lists, without repeating tool output.
2. Do not read output JSON, images or fixtures unless the task requires it; read only the part of a file that is needed and do not re-read what was just written.
3. At most 3 subagents per round, passing each one only its file and its data, not the conversation history.

B Reference repository layout

Path	Purpose
AGENTS.md / CLAUDE.md	Agent rules (kept identical, or one imports the other)
state/pending.json	Generated shared task state
state/config.json	Project settings: code folders, critical paths, minimum mutation score
state/tasks/task_NNN.json	One task file per task, written by the agent
context/global/, context/roles/*/	Generated last-context files
context/tasks/task_NNN_context.json	One delta per task, written by the agent
scripts/merge_state.py	Regenerates the shared state in CI
scripts/check_task_delta.py	Rejects code changes without a valid delta
scripts/mutation_targets.py	Lists mandatory and optional files to mutate, and the minimum score
.github/workflows/	CI with path filters and parallel jobs

Table 4. Reference layout of the boilerplate.